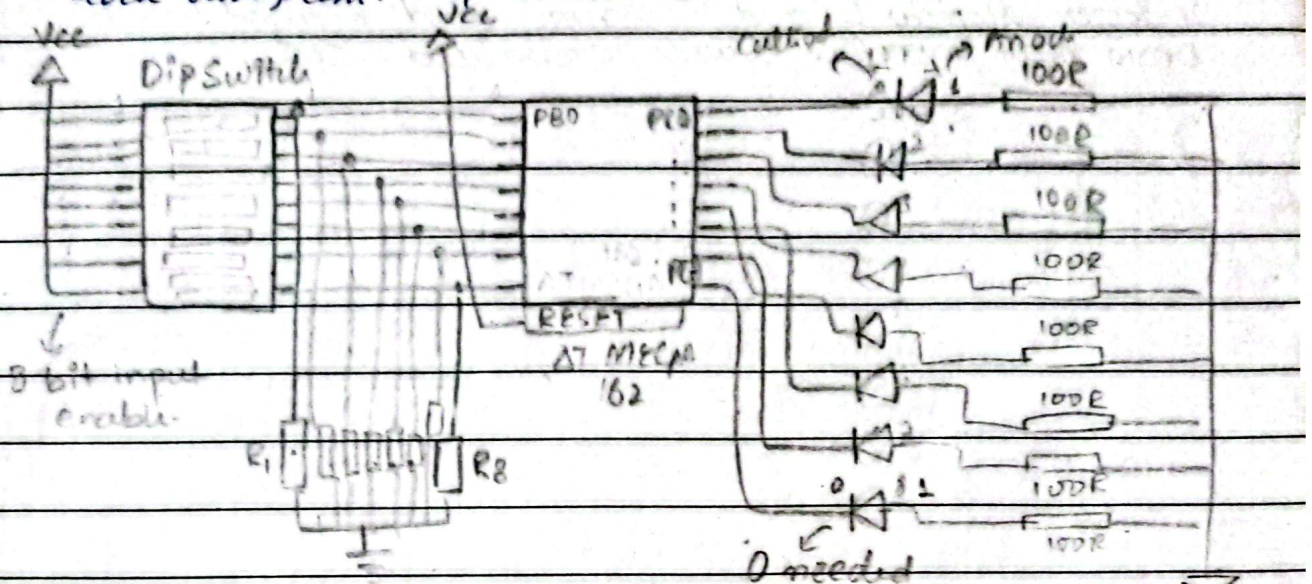


Date: _____

Q₁ Design a dip switch that takes 8-bit input from dip switches through port B. performs a bit wise or operation with a mask 10001111 and displays the result on a common anode LED bank through port C. Draw neat interface block diagram and write the embedded C code.



```
#include <avr/io.h>
```

```
#define F_CPU 1000000UL
```

```
#include <util/delay.h>
```

```
int main(void)
```

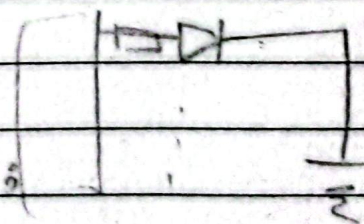
```
{ DDRC = 0b 11111111 // port C = output
  while (1)
```

```
{ input-data = PINB;
```

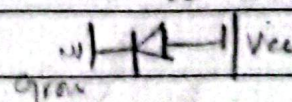
```
  result = input-data | 0b 10001111;
```

```
  PORTC = ~result; // common anode
```

```
} }
```



For led to work

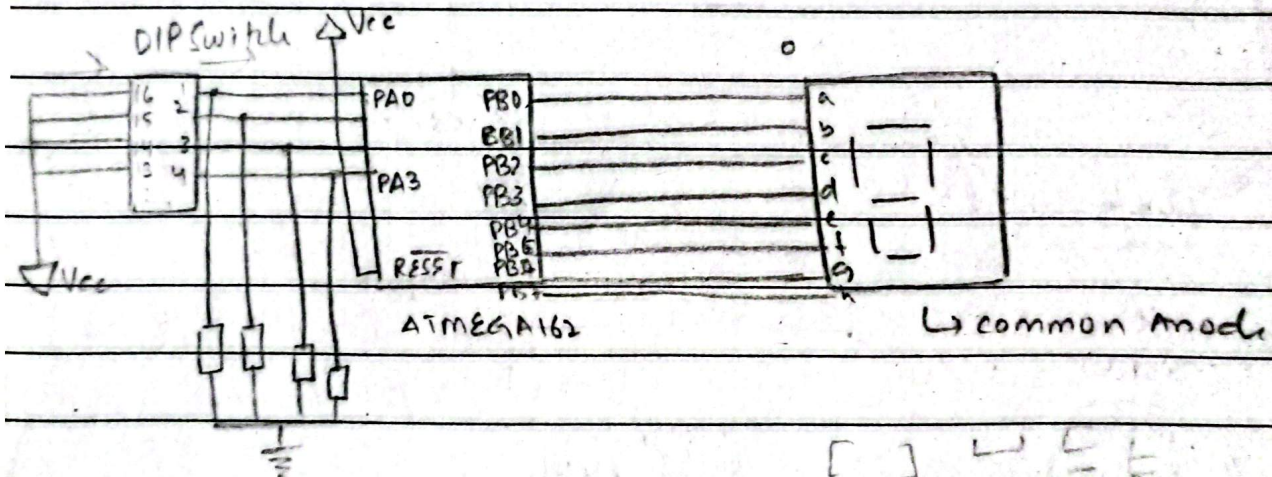
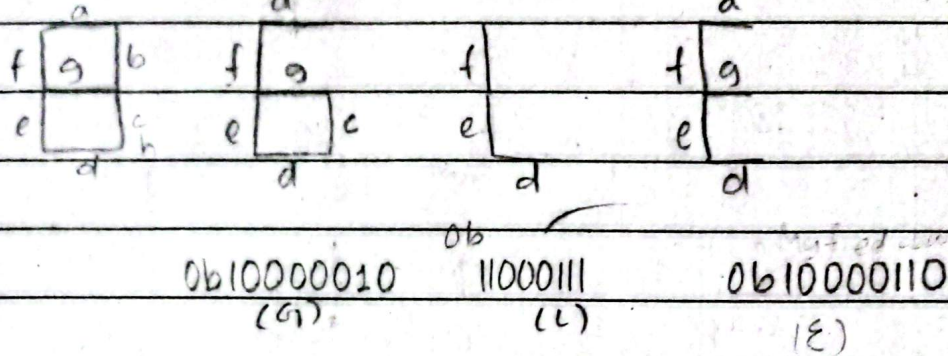


To display:
PORTC = value

To read
var = PINA

Date: _____

Q3. Design a circuit that takes 4bit operand num from dip switches through lower bits of PORTA. and performs comparison operation with values A16, if num is greater than A16; letter G should be displayed. on common anode 7 segment display. if lesser display L and if equal E displayed. Draw neat interface & write embedded C code.



#include <avr/io.h>

#define F_CPU 1000000UL

#include <util/delay.h>

int main(void) {

DDRA &= 0b11110000; or 0b00000000 → take all as input

DDRC = 0b11111111;

while(1) {

num = PINA & 00001111;

if (num > 0b00001100) {

PORTC = 0b10000010;

elseif (num < 0b00001100) {

PORTC = 0b11000111;

else {

PORTC = 0b10000110;

-delay-ms(1000)

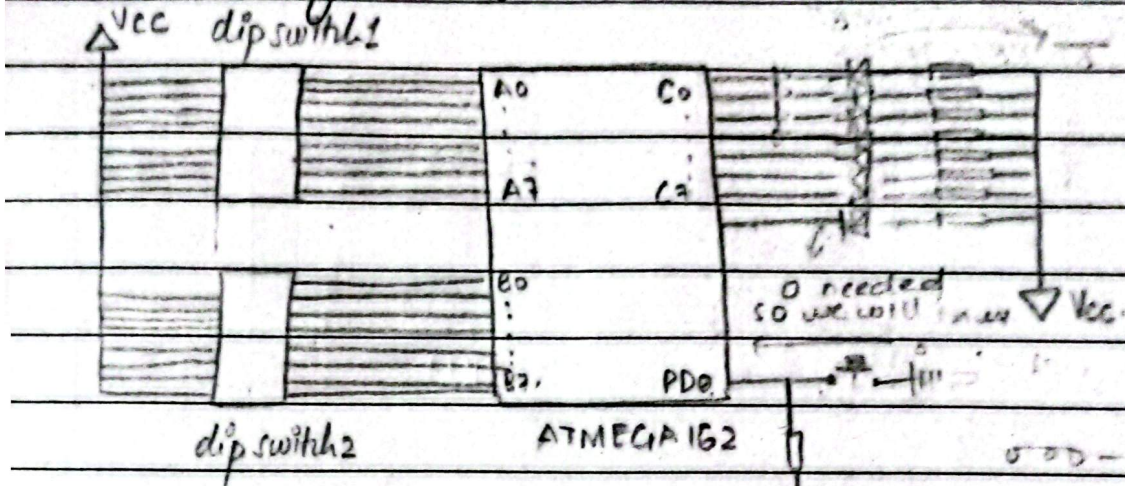
}

Page No. _____

How to connect push button

Date: _____

Q5. Design a circuit that takes two 8-bit operands num1 and num2 from 2 dip switches and display 16-bit result alternately on LED bank with reasonable delay until a push button called 'stop' is pressed once. Configure the push button as active low input. Draw a neat interface diagram & write embedded C-code.



```
#include <avr/io.h>
int main(void) {
    DDRA = 0b00000000; //input
    DDRB = 0b00000000; //input
    DDRC = 0b11111111; //output
    DDRE = 0b00000001; //output
    PORTD = (1<<0); //pull up enabled
    stop_flag = 0;
    while(1) {
        num1 = PINA;
        num2 = PINB;
        if (stop_flag)
            continue;
        PORTC = num1;
        delay_ms(1000);
        PORTC = num2;
        delay_ms(1000);
        if (~(PIND)) {
            stop_flag = 1;
            continue;
        }
        PORTC = num1;
        delay_ms(1000);
        PORTC = num2;
        delay_ms(1000);
        if (~(PIND)) {
            stop_flag = 1;
            continue;
        }
    }
}
```

Annotations:

- $DDRD = (1 \ll 0)$ → only bit 0
- $PORTD = (1 \ll 0)$; $xxxxxx1$ ⇒ pull up enabled
- enabled PORTD pull up. So when push button not connected $PIND = 1$ (High) ⇒ hence no stop.
- because we want active low

Media Code: No AVE, nuds till keypad.
 1 Q → str. Unligned.

Need solution.

Date: _____

Q6. Using BCD decoder IC 7447 (active low outputs for segments a through g interface a common cathode 7 segment display, to display any number from 0 through 9 from dip switch.

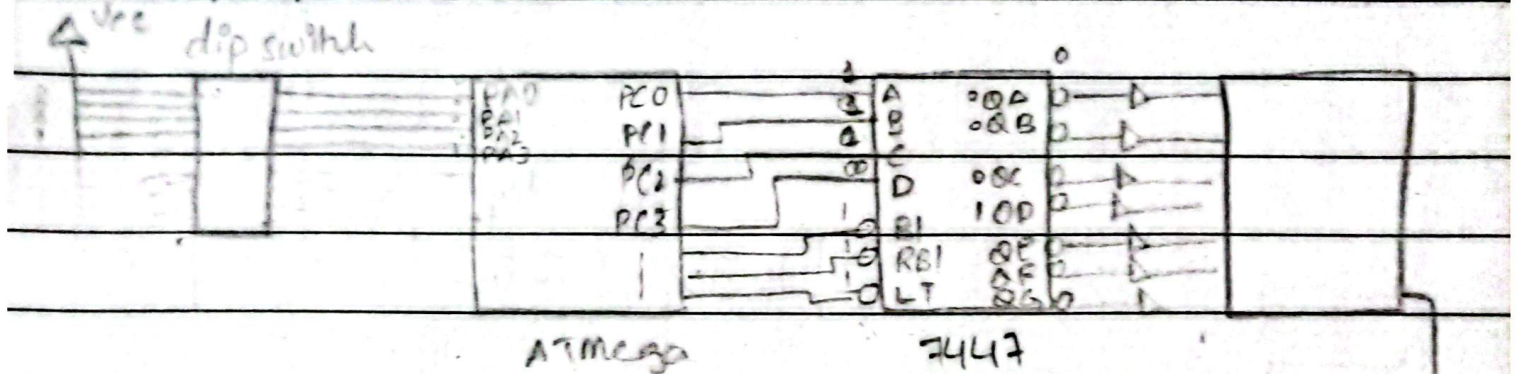
Diagram + Code.

DCBA → active high

BCD input → D, M, B, A

LT, BI, RBI: active low.

How to handle common Cathode



int main(void)

DDRA = 0b 00000000; A = Input

DDRC = 0b 11111111; C = Output

PORTC = PINA | 0b 11110000;

PORTA = 0b 11111111;

while (1)

{ PORTC = PINA & 0b 00001111;

}

}

1 to glow
 1 to glow

from dip switch 1111 1111 Port A
 1111 0000

1111

to display
 BI/RBI/LT

DCBA =
 F3, F2, F1, F0

If I give 1000
 so now 0111 will

xxxx 0111
 1111 0000

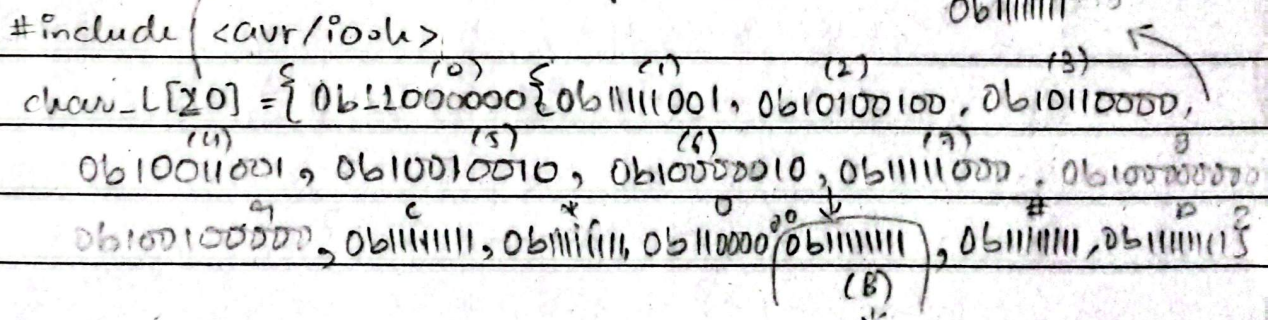
1111 0111

A = xxxx 1101
 0000 1111

key - 90 dr

Date: _____

Q1. Interface ATmega162 with keypad through keypad controller IC and display the digit 0 through 9 on a common anode 7-segment display. Write embedded C code.



→ Have put 0xFF in place of not needed values, so that it'd premed, won't run.

```
DDRB = (1 << PBO); // only PBO = 1 (output) (enable)
```

Print key:

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Date: 20/04/20

int GetKeyPressed();

12/04/20

{ char x;

PORTB = 0x00; // clear PORTB as char[16] = 0b11111111

int data = 16; // default dont glow led

x = ANC;

if (x == 0x01){

data = (PIND & 0x0F);

return data;}

return 16

while(1){

key = GetKeyPressed();

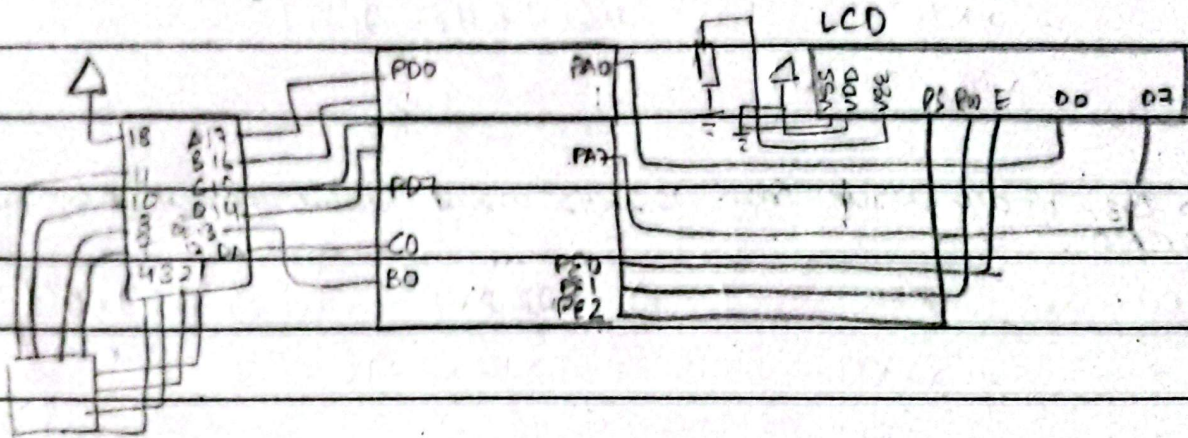
if ((key != 16) & digit[key] != 0b11111111){

PORTA = char_1[key]

}

Date: _____

Q2. Interface ATmega162 with 4x4 keypad through keypad controller IC 74922 & display the characters on the keypad keys to a 16x2 LCD in the second line starting at address 44hex. You may assume a user def. func.
 - set-LCD-address(char-add-instr) that changes the address by instr. (DB7-DB0) = 0b1AAAAAAA. No need to write user def. funcs. only write embedded C main func when you call the above pr.



```
void LCD_init(void);          void LCDclear
```

```
void LCD_write_clear(char);
```

```
void set-LCD-address(char-add-instr);
```

```
char key.map[16] = {'1', '2', '3', 'A', '4', '5', '6', 'B', '7', '8', '9', 'C',  
                   '0', '#', 'D'};
```

```
int main(void){
```

```
    DDRD = 0xF0;
```

```
    DDRC = 0x00;
```

```
    DDRB = (1 << PB0);
```

```
    DDRA = 0xFF;
```

```
    DDRE = 0xFF;
```

```
    ...
```

```
    -LCDclear();
```

```
    -set-LCD-address(0x44)
```

```
    while(1){
```

```
        PORTB = 0x00;
```

```
        if PIND == 0x01{
```

```
            key = PIND & 0x0F
```

```
            if (key != 16){
```

```
                PORTA = key.map[key]
```

```
        }
```

```
    }
```

$\sim$ & $\rightarrow$ to clear

$1 \rightarrow$ to change a bit

$\wedge$ (XOR) $\rightarrow$ to toggle a bit

Date: _____

Q3. For the timer/counter0 in CTC mode, if the microcontroller clock freq. is 8MHz. what will be the output freq. at OCR0 if prescaler is set to 16 and OCR0 holds 149.

$$f_{CTC} = \frac{f_{clk}}{2 \times N \times (1 + OCR0)}$$

$f_{clk} = 8\text{MHz}$ (freq. of MCU)
 $N = 16$ (prescaler)
 $OCR0 = 149$

$$= \frac{8 \times 10^6}{2 \times 16 \times (1 + 149)} \Rightarrow 1.667 \text{ KHz} = f_{CTC}$$

Q4. In Q3, if o/p wave freq. is 2KHz what value should be in OCR0.

$$OCR0 = \frac{f_{clk}}{f_{CTC} \times 2 \times N} - 1 \Rightarrow \frac{8 \times 10^6}{2 \times 10^3 \times 2 \times 16} - 1 \Rightarrow 124 = OCR0$$

Q5. Set bit positions of reg. PORTA without affecting the rest of the bits. $\rightarrow$ sets only bit 5.

$$PORTA \mid = (1 \ll 5)$$

Q6. Reset (clear) bit 2 of DDRD without affecting other bits.

$$DDRD \&= \sim (1 \ll 2)$$

// 1 = 00000001
// 1 shift = 00000100
// NEG = 11111011
and 01010100

let say DDRD = 01010101

0101000
 $\rightarrow$ clear

Q7. Toggle bit 7 of PORTE (without affecting other bits) $= (1 \ll 7)$

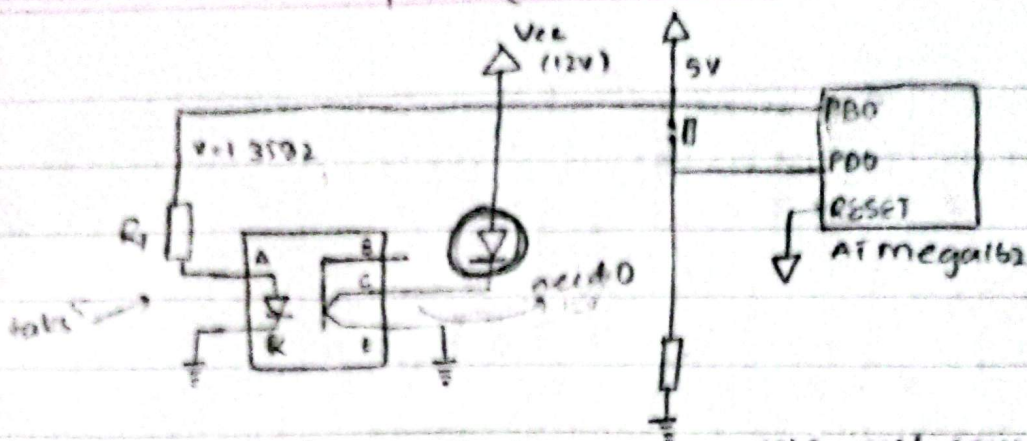
$$PORTE \wedge = (1 \ll 7)$$

$x \rightarrow$ the bit to be changed

$$porta = 01010101 \quad \text{or} \quad 01011010$$

1111

Lab14 Opto Coupler. (Practice Problem Lab14)



```
#define include <avr/io.h>
#define F_CPU 1000000UL
#include <util/delay.h>
```

We just say, turn the LED on by just giving any 5V signal on pin A of optocoupler

```
int main(void){
    DDRD = 0x00; D → input
    DDRB = 0xFF; B → output
    char y;
    PORTB = 0; // optocoupler's LED off
    while(1){
        y = PIND;
        if(y == 1){
            PORTB ^= (1<<0); // toggle bit 0.
            // shift 0000 0001 0 times
            _delay_ms(1000);
        }
    }
}
```

LEC 28

slide 2:

MCU clock = 8MHz

$$\Rightarrow \frac{8 \times 10^6}{1024} \Rightarrow 7812.5 \text{ Hz}$$

Timer increment is every 7812.5 times in a second. (count+1)
 $f_{\text{out}} = \frac{\text{count}}{\text{se}}$

Timer Prescaler = 1024

256 clock ticks = 8 bit timer $\rightarrow 2^8 = 256$

1 clock tick $\rightarrow \frac{1}{7812.5} \text{ s}$

256 clock ticks $\rightarrow \frac{256}{7812.5} = 32.78 \text{ ms}$

count = 256

256 clock ticks = one overflow takes 32.78 ms.

Q: How many overflows in second?

1 overflow $\rightarrow 32.78 \text{ ms}$

$\frac{1}{32.78 \text{ ms}}$ overflows $\rightarrow$ in one second

30.5 overflows $\rightarrow$ in one sec

$\wedge$ = Toggle
 $\mid$ = set
 $\sim$ = set 0

```

int main(void) {
  DDRB |= (1 << PBO);
  TCCR0 = 0x00; Normal mode
  TCCR0 |= (1 << CS00) | (1 << CS01) Prescaler 1024
  TIMSK |= (1 << TOIE0);
  sei();
  while(1) { }
}

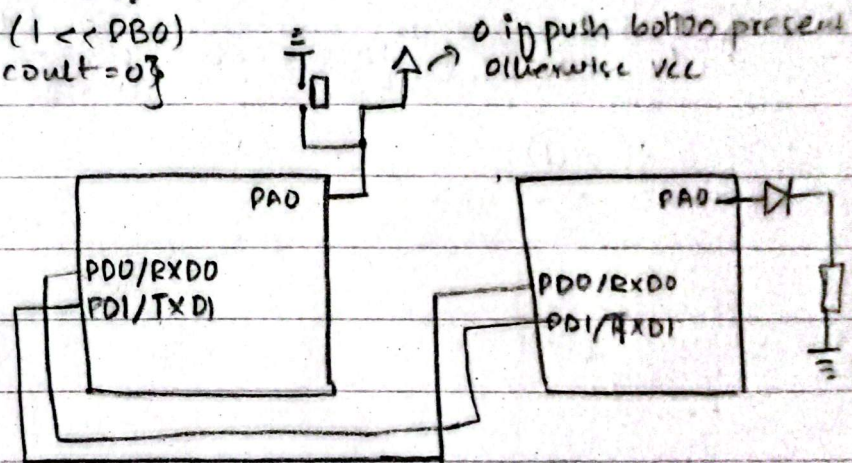
```

```

ISR(Timer0_OVF_vect) {
  overflow-count++;
  if (overflow-count > 31) {
    PORTB ^ = (1 << PBO);
    overflow-count = 0;
  }
}

```

Lab 11:



Transmitter:

```

int main(void) {
  set DDRD |= (1 << PD1); TX
  set DDRD |= (~(1 << PD0)); RX
  pullup - PORTD |= (1 << PD0); 1 on no input receive pin
  DDRA = 0x00;
  set [ UBBR0H = 0x00;
  [ UBBR0L = 0x00;
  give [ UCSRB = (1 << TXEN0)
  to Q [ UCSRC =
  [ char y = 0
  [ while(1) {
  [ y = PINA & 0b00000001;
  [ if (y == 0) {
  [ while (! (UCSRB & (1 << UDRE0)))
  [ UDRO = 0b
  [ delay;
  [ }
}

```

PRACTICE PROBLEM 4 → Q5

```
#define F_CPU 8000000UL
```

```
int main(void){
    DDRD |= (1<<PDI); PDI = 17x0
    DDRDB |= (1<<PDI); RXD
    USART_init()
    sei()
    while(1){
        ISR handling??
    }
}
```

```
int USART_init() {
    UERR0H = 0x00;
    UERR0L = 0x00;
    UCSRB |= (1<<TXEN0)|(1<<RXEN0)|(1<<RXC1E0);
    UCSRC |= (1<<URSEL) | (3<<UCS200);
}
```

```
int USART_Transmit(char A){
    while((UCSR0B & (1<<UDRE0)) == 0){
        UDRO = A;
    }
}
```

```
ISR(USART_RX_vect){
    data = UDRO;
    USART_Transmit(data);
    if (data == 'A'){
        PORTB ^= (1<<PBI);
    }
}
```

Q2: int main(void){

```
    DDRD |= (1<<PDS); PDS = output
```

```
    TCRO = 0x00;
```

```
    TCRO |= (1<<CS02)|(1<<CS00)|(1<<CS01); prescale
```

```
    TIMSK |= (1<<TOIE0);
```

```
    sei();
```

```
    while(1){
        //
    }
}
```

```
ISR(Timer0_OVF_vect){
```

```
    overflow_count++;
```

```
    if (overflow_count > 122){
```

```
        PORTD ^= (1<<PDS)
```

```
        overflow_count = 0;
    }
}
```

$$\frac{8000000}{256} = 31250 \text{ Hz} \cdot \frac{\mu\text{s}}{\text{s}}$$

$$1 \text{ Hz} \rightarrow \frac{1}{\text{s}}$$

$$256 \text{ Hz} \rightarrow \frac{256}{31250} = 8.192 \times 10^{-3}$$

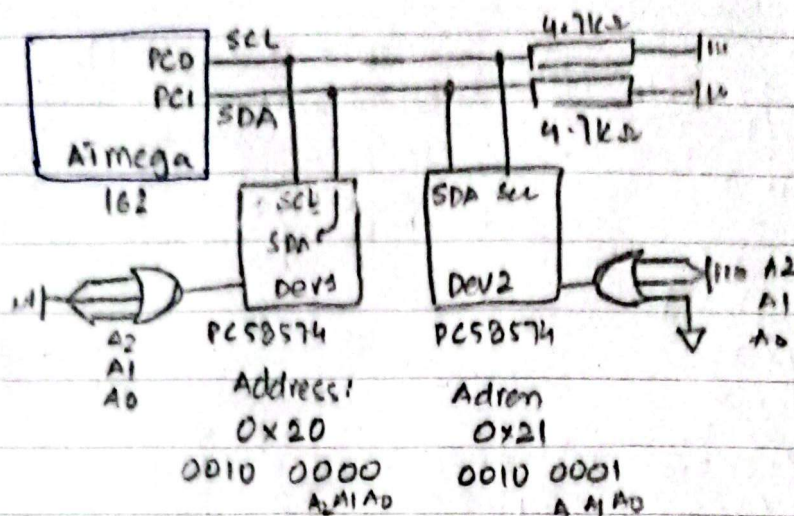
$$1 \text{ of} \rightarrow 8.19 \text{ ms}$$

$$1 \text{ s}$$

$$8.19 \text{ ms}$$

$$15 \rightarrow 122 \text{ overflow}$$

PP4 → Q52c.



Q3. C101.

(i)

(V)	Voltage	2.5V	3V	3.5	4
nJ	Energy	19.53	28	38.28	50nJ
MHz	Frequency	125MHz	150	175	200
	CycleTime	8	6.67	5.71	5.5

Target $t = 50 \times 10^9$ cycles $t = 5.5 \text{ min} = 5.5 \times 60 = 330 \text{ s}$

$$E_2 = \frac{V_2^2}{V_1^2} \cdot E_1$$

For 3.5V:

$$E_2 = \frac{(3.5)^2}{4^2} \cdot 50 \text{ nJ}$$

$$= 38.28 \text{ nJ}$$

For 3V:

$$E_2 = \frac{3^2}{4^2} \cdot 50 \text{ nJ}$$

$$= 28 \text{ nJ}$$

For 2.5V:

$$E_2 = \frac{2.5^2}{4^2} \cdot 50 \text{ nJ}$$

$$= 19.53 \text{ nJ}$$

For 3V

$$f_2 = \frac{3}{4} \times 200 \text{ MHz}$$

$$= 150 \text{ MHz}$$

For 3.5V

$$f_2 = \frac{3.5}{4} \times 200 \text{ MHz}$$

$$= 175 \text{ MHz}$$

For 2.5V

$$f_2 = \frac{2.5}{4} \times 200 \text{ MHz}$$

$$= 125 \text{ MHz}$$

$$T = \frac{1}{f}$$

Max Performance → choose highest f

Min Energy → choose lowest f

$E \propto V^2 \propto f$

↓ ↓

(ii) Maximum Perf.
choosing highest f.

Running 500 MHz for 250s

$$t_1 \leq 330 \quad \cdot \quad 250 \leq 330$$

$$280 \times 10^6 \times 250 = 50 \times 10^9 \text{ cycles}$$

via hit & try

" cycles completed we cannot
have any other combo
with this

$$E = 50 \times 10^9 \times 50 \times 10^9$$

$$E = 2500 \text{ J}$$

(iii) choosing lowest f:

→ lowest 2.5V → 125 MHz gives

41.25 × 10⁹ cycles in 330s.

Therefore not using.

→ let's go with 3V → 150 MHz → t₁

$$t_1 + t_2 = 330 \Rightarrow t_1 = 330 - t_2$$

eq (2)

$$150 \times 10^6 \times t_1 + 125 \times 10^6 \times t_2 = 50 \times 10^9$$

$$150 \times 10^6 (330 - t_2) + 125 \times 10^6 \times t_2 = 50 \times 10^9$$

$$-150 \times 10^6 t_2 + 125 \times 10^6 t_2 = 500 \times 10^6$$

" -ve not using 2.5V

try 175 MHz

$$150 \times 10^6 \times t_1 + 175 \times 10^6 \times t_2 = 50 \times 10^9$$

$$-150 \times 10^6 t_2 + 175 \times 10^6 t_2 = 500 \times 10^6$$

$$25 \times 10^6 t_2 = 500 \times 10^6$$

$$t_2 = 20$$

$$t_1 = 330 - 20 \Rightarrow t_1 = 310$$

→ 3V for 310s:

$$\text{cycles} = 310 \times 150 \times 10^6 \Rightarrow 4.65 \times 10^{10}$$

$$E = 4.65 \times 10^{10} \times 28 \times 10^9$$

$$E_1 = 1302 \text{ J}$$

→ 3.5V for 20s:

$$\text{cycles} = 20 \times 175 \times 10^6 = 3500 \times 10^6$$

$$E = 3500 \times 10^6 \times 38.28 \times 10^9$$

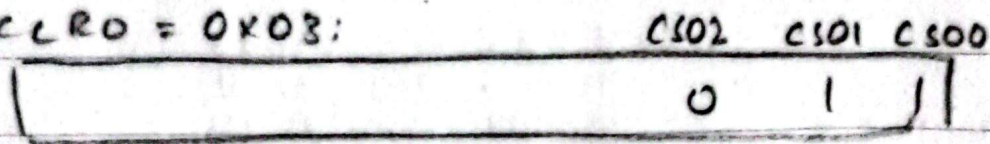
$$E_2 = 13398 \text{ J}$$

$$E = E_1 + E_2 = 1435.98 \text{ J}$$

$$C_1 + C_2 = 5 \times 10^9$$

Q: TCCR0;

TCCR0 = 0x03;



From table: prescaler = 64 = N

$$f_{\text{overflow}} = \frac{F_{\text{CPU}}}{N \times 256} = \frac{1 \times 10^6}{64 \times 256} = 61.035 \text{ kHz}$$

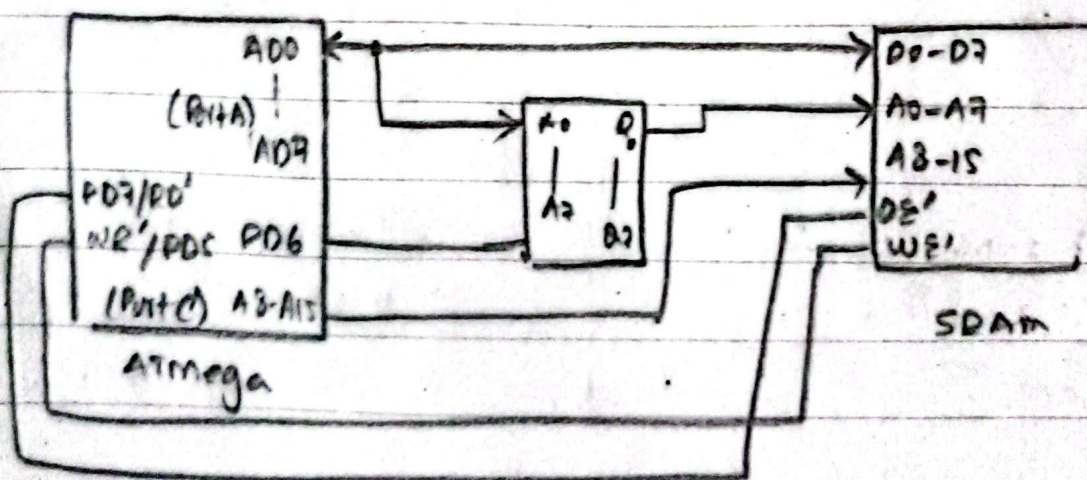
↓
overflows in one second

$$T = \frac{1}{f} = \frac{1}{61.03} = 1.638 \text{ ms}$$

↪ one overflow takes 1.638 ms

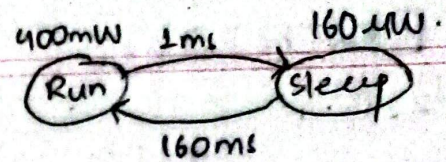
Q: ATmega162 with 64KB external RAM

$$64 \text{ KB} = 64 \times 1024 = 65536 \Rightarrow 2^{16}$$



PAST PAPER

2023 → Batch 2020 → Q3.



20^a, 250^L, 200^a, 60^L, 40^a, 2000^L, 50^a, 150^L, 1500^a ms

(i) Fixed Timeout with a timeout of 60 ms ^{60 = 60 (no benefit of sleep)}

$$(20+60)^a + 1ms + (250-60)^L + 160 + (200+60+40+60)^a + 1 + (2000-60)^L + 160 + (50+60)^a + 1 + (150-60)^L + 160 + 1500$$

→ Trs as run time

$$\text{Execution Time} = 2050^a + (160 \times 3)^{Tr} + (1 \times 3)^{Tr} + 2220^L$$

$$= 4.753s$$

$$\text{Energy} = (2050^a \times 400m) + (480m \times 400m) + (3 \times 400m) + (2220^L \times 160m)$$

$$= 1.01J$$

(ii) Fixed Timeout with timeout of 500ms.

$$(20+250+200+60+40+500)^a + 1 + (2000-500)^L + 160 + (50+150+1500)^a$$

$$ET = 1070^a + 1 + 1500^L + 160 + 1700^a \Rightarrow 4.431s$$

$$E = [(1070^a + 1700^a) \times 400m] + (1^a \times 400m) + (160^L \times 400m) + (1500^a \times 160 \times 10^6) = 1.172J$$

(iii) Predictive Threshold 60 ms:

$$20^a + 1 + 250^L + (200+60+40)^a + 1 + (2000)^L + 160 + 50^a + 1 + 150^L + 160$$

$\downarrow$ 40 < 60 $\downarrow$ < 60

$$ET: 2353^a + 2400^L = 4.753s$$

$$E = (2353^a \times 400m) + (2400^L \times 160 \times 10^6) = 0.94J$$

(i) Performance critical → choose len time = (ii) 4.43s

(ii) Energy constraint → len energy (iii) = 0.94J